

fms

Transformers

Dr. Abdallah Salem Hyassat

Email : Abdallah.s.Hyassat@gmail.com

LinkedIn : [Abdallah Salem Hyassat](#)

Motivation

- To motivate the transformer, consider the following passage: *ex:*

The restaurant refused to serve me a hamburger sandwich because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambiance was just as good as the food and service.

- The goal is to design a network to process this text into a representation suitable for downstream tasks.
- For example, it might be used to classify the review as positive or negative or to answer questions such as “Does the restaurant serve steak?”

Motivation

- We can make **three** immediate observations

1- • First, **the encoded input can be surprisingly large.**

- **In this case**, each of the 37 words might be represented by an embedding vector of length 1024, so the encoded input would be of length $37 \times 1024 =$
ex: 37888 even for this small passage.

- A more realistically **sized body** of text might have **hundreds or even thousands of words**, so **fully connected neural networks are impractical.**

the problem:

2- • Second, one of the defining characteristics of NLP problems is that **each input (one or more sentences) is of a different length**; hence, **it's not even obvious how to apply a fully connected network.**

Motivation

• Problem of Standard fully-connected layer

- 1- • A very large number of parameters
- 2- • Can't cope with text of different lengths

• Conclusion:

- These observations suggest that the network should (share parameters) across words at different input positions, similarly to how convolutional networks share parameters across different image positions.
- So, We need a model where parameters don't increase with input length

Motivation

3-• Third, language is ambiguous

The restaurant refused to serve me a hamburger sandwich because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambiance was just as good as the food and service.

- It is unclear from the syntax alone that the pronoun **it** refers to the **restaurant** and not to the **hamburger** sandwich. To understand the text, the word **it** should somehow be connected to the word restaurant
- Moreover, these connections need to extend across large text spans. For example, the word **their** in the last sentence also refers to the **restaurant**.
- **Conclusion**:
 - 1-• There must be connections between the words.
 - 2-• The strength of these connections will depend on the words themselves

imp

Transformers

definition:

- Transformers are neural networks that learn context and understanding through sequential data analysis.
- The Transformer models use a modern and evolving mathematical techniques set, generally known as attention or self-attention. what does the model depends on :
- This set helps identify how distant data elements influence and depend on each other. (attention) (self-attention)
- Transformers came into action in a 2017 Google paper as one of the most advanced models ever developed. This has resulted in a wave of advances called "Transformer AI" in machine learning.
- Transformer outperforms both (recurrent and convolutional models) on academic English to German and English to French translation benchmarks.

ex:

*Attention Mechanism:

- To understand transformers we first must understand the attention mechanism.
- The Attention mechanism enables the transformers to have extremely long term memory.
- A transformer model can “attend” or “focus” on all previous tokens that have been generated.

imp

bing's answer :

what is the difference between (RNN) and (TRANSFORMERS) :

- RNNs are sequential models that process data one element at a time, maintaining an internal hidden state that is updated at each step. They operate in a recurrent manner, where the output at each step depends on the previous hidden state and the current input.
- Transformers are non-sequential models that process data in parallel, using a technique called self-attention to encode the input sequences. They do not rely on recurrence or hidden states, but instead use positional embeddings to capture the order of the input elements.
- RNNs are effective in capturing short-term dependencies in sequences, but they suffer from the vanishing gradient problem, which makes it difficult to learn long-term dependencies. Transformers overcome this problem by using self-attention, which allows them to directly access any element in the input sequence, regardless of its distance.
- RNNs are slower to train and infer than Transformers, because they have to process the input data sequentially, which limits the parallelization. Transformers are faster and more scalable, because they can process the input data in parallel, using matrix operations and hardware acceleration.

Example of using Transformer

- Let's walk through an example. Say we want to write a short sci-fi novel with a generative transformer.
- Our input: "As Aliens entered our planet".
- Transformer output: "and began to colonized Earth, a certain group of extraterrestrials began to manipulate our society through their influences of a certain number of the elite to keep and iron grip over the populace."
- As the model generates the text word by word, it can "attend" or "focus" on words that are relevant to the generated word.
- The ability to know what words to attend too is all learned during training through backpropagation.

imp

2/2/21

As aliens entered our planet



RNN

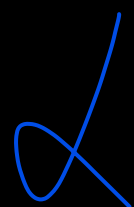
what is the difference between (RNN) and (TRANSFORMERS) :



The ANSWER of the Q from the slides

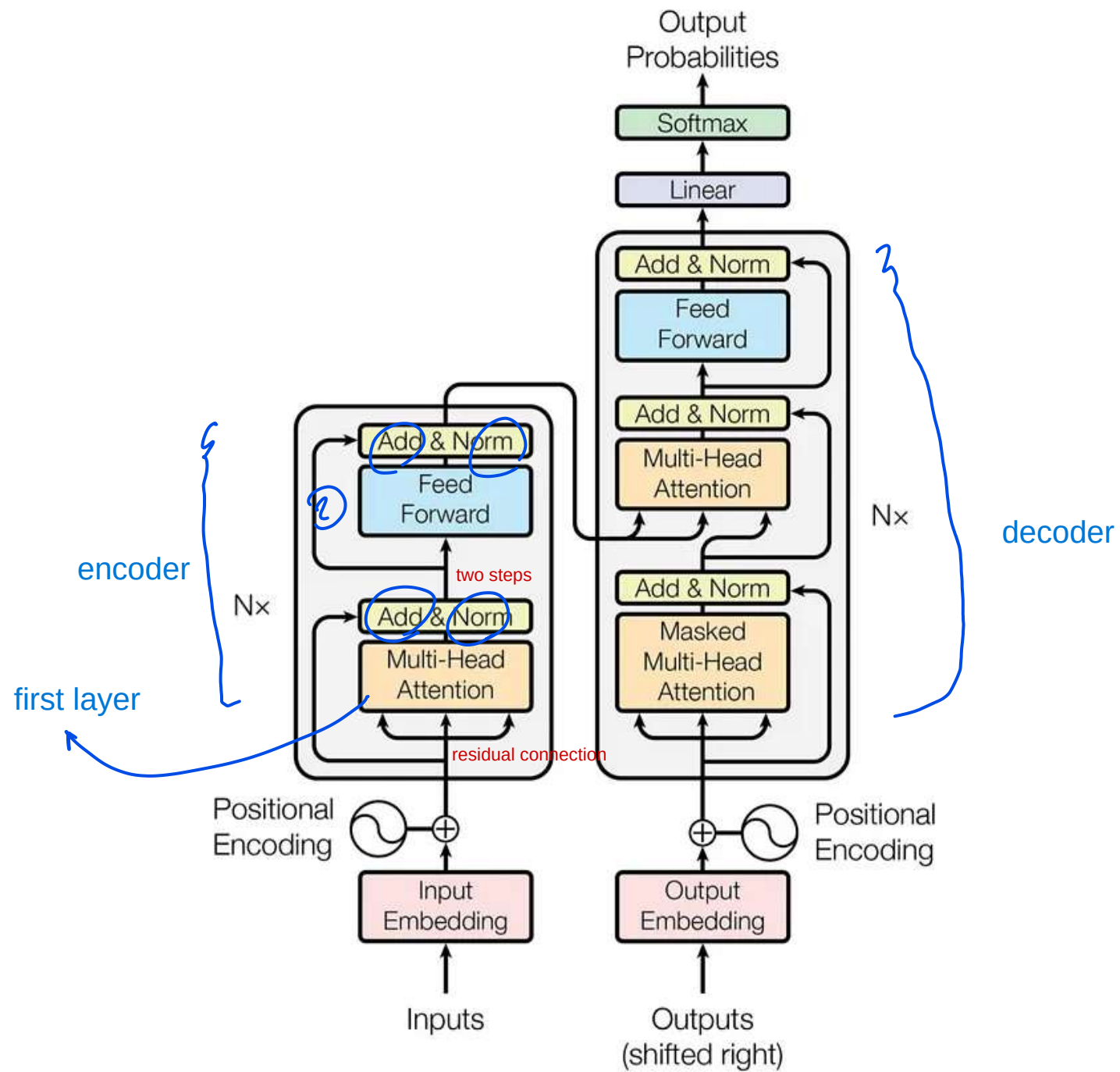


- Recurrent neural networks (RNN) are also capable of looking at previous inputs too.
- But the power of the attention mechanism is that it doesn't suffer from short term memory.  AKA: MAHDOOD explanation:
- RNN's have a shorter window to reference from, so when the story gets longer, RNN's can't access words generated earlier in the sequence.
- This is still true for Gated Recurrent Units (GRU's) and Long-short Term Memory (LSTM's) networks, although they do a bigger capacity to achieve longer-term memory, therefore, having a longer window to reference from.
- The attention mechanism, in theory, and given enough compute resources, have an infinite window to reference from, therefore being capable of using the entire context of the story while generating the text.



Attention Mechanism has an infinite reference window

As aliens entered our planet and began to colonize earth a certain group of extraterrestrials ...



Transformer

- On a high level, the encoder maps an input sequence into an abstract continuous representation that holds all the learned information of that input.
↳ latent space
- The decoder then takes that continuous representation and step by step generates a single output while also being fed the previous output.

How do transformer models work?

1. Input embedding

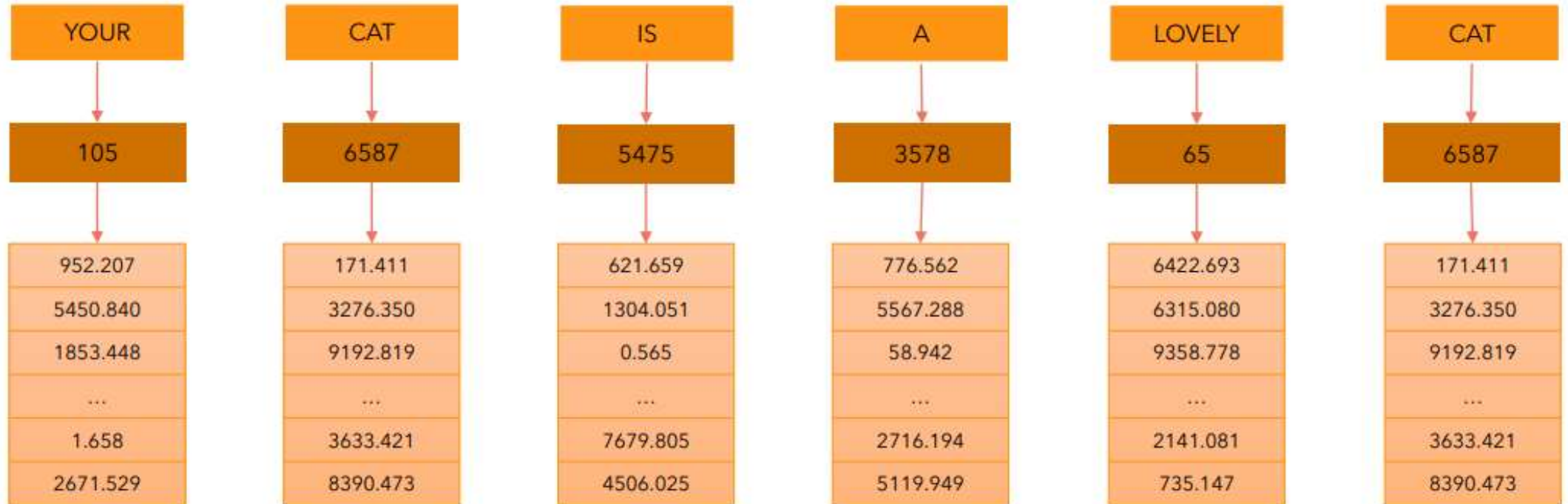
- The first step in transformer operations is understanding the input data. It takes a sentence — or a sequence of data — broke it down into a series of tokens and turns each word or element into numerical representation known as vector embeddings.
- Various techniques can be employed for input embedding, such as word embeddings and character embeddings.
- This allows the model to work with continuous representations rather than discrete symbols.

Input embedding

Original sentence
(tokens)

Input IDs (position in
the vocabulary)

Embedding
(vector of size 512)



We define $d_{\text{model}} = 512$, which represents the size of the embedding vector of each word

How do transformer models work?

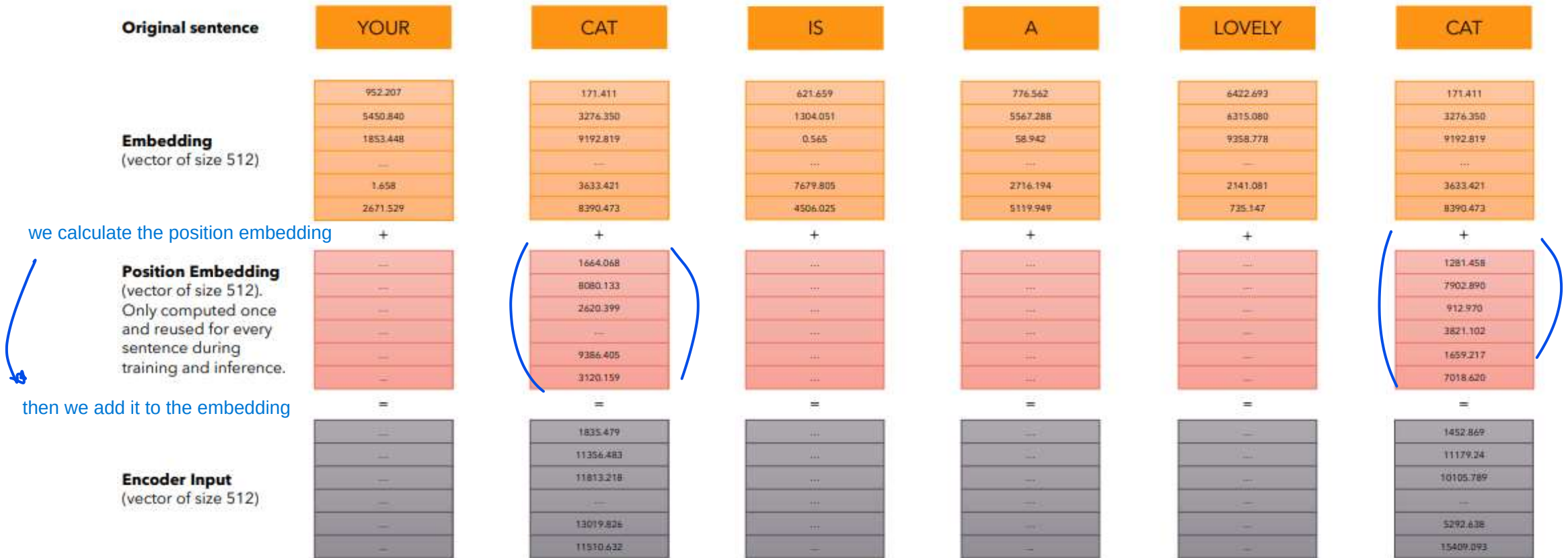
2. Positional encoding

the importance of the positional encoding

- Transformers don't naturally understand the order of words, so they use positional encoding to give the model information about the order.
- We want each word to carry some information about its position in the sentence.
- We want the model to treat words that appear close to each other as "close" and words that are distant as "distant".
- This is done by combining the embeddings with sinusoidal functions, which helps the model understand the relationships between parts of the sequence.

Positional Encoding

the cats here are not the same because every one of them has different positional embedding



Calculate Positional Encoder

①
row, column
سطر، ستون

$$PE(pos, 2i) = \sin \frac{pos}{10000^{\frac{2i}{d_{model}}}}$$

the length of the vector

②
row, column
سطر، ستون

$$PE(pos, 2i + 1) = \cos \frac{pos}{10000^{\frac{2i}{d_{model}}}}$$

We only need to compute the positional encodings once and then reuse them for every sentence, no matter if it is training or inference.

Sentence 1

YOUR

PE(0, 0)
PE(0, 1)
PE(0, 2)
...
PE(0, 510)
PE(0, 511)

CAT

PE(1, 0)
PE(1, 1)
PE(1, 2)
...
PE(1, 510)
PE(1, 511)

IS

PE(2, 0)
PE(2, 1)
PE(2, 2)
...
PE(2, 510)
PE(2, 511)

Sentence 2

I

PE(0, 0)
PE(0, 1)
PE(0, 2)
...
PE(0, 510)
PE(0, 511)

LOVE

PE(1, 0)
PE(1, 1)
PE(1, 2)
...
PE(1, 510)
PE(1, 511)

YOU

PE(2, 0)
PE(2, 1)
PE(2, 2)
...
PE(2, 510)
PE(2, 511)

Encoder

3. Encoder layers

- The embedded and encoded input sequence is passed through multiple encoder layers. Each layer consists of two sub-layers called the self-attention mechanism and the feed-forward neural network.
- 1- The self-attention mechanism allows the model to focus on different parts of the input sequence and capture dependencies. It calculates attention scores (for each element based on its relationships with other elements in the sequence).
 - For each word in a sentence, the self-attention layer computes three vectors (key, value, query). To determine a word's contextually related words, the dot products of the query vector are considered with the key vectors of the other words.
 - 2- The feed-forward neural network applies a non-linear transformation to the outputs of the self-attention mechanism, (introducing complexity and expressive power to the model.) The feed-forward layer makes up two-thirds of the parameters in a transformer model.

Query, Key and Value

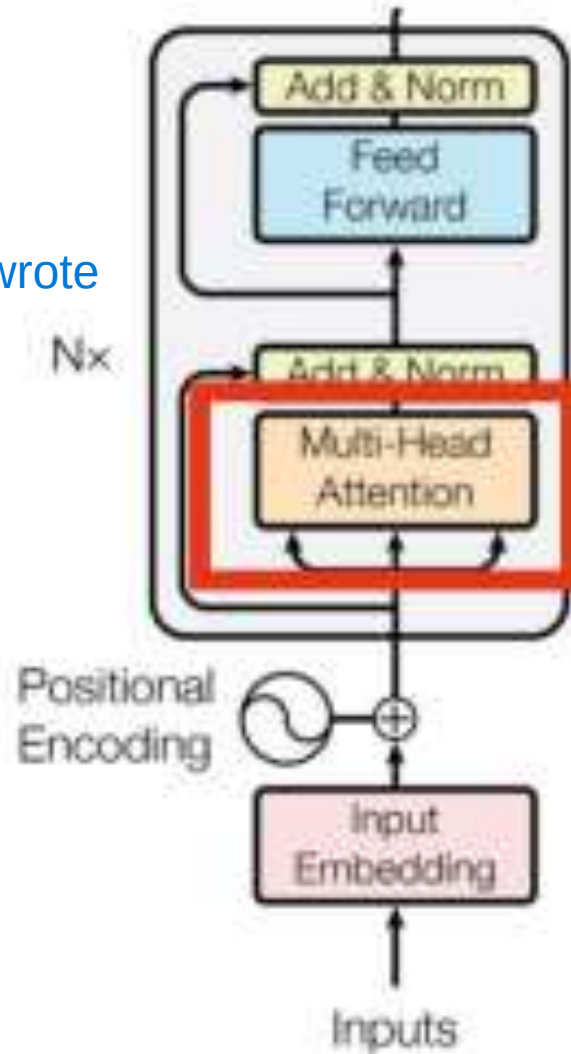
- 1-• **Query (Q):** The query matrix is obtained by multiplying the input embeddings (or the outputs of the previous layer) by a learned weight matrix specific to the attention mechanism. This results in a matrix that serves as the queries for the attention mechanism.
- 2-• **Key (K):** Similarly, the key matrix is obtained by multiplying the input embeddings (or the outputs of the previous layer) by another learned weight matrix specific to the attention mechanism. This matrix serves as the keys for the attention mechanism.
- 3-• **Value (V):** The value matrix is obtained by multiplying the input embeddings (or the outputs of the previous layer) by yet another learned weight matrix specific to the attention mechanism. This matrix serves as the values for the attention mechanism.

the difference is that every one of them is multiplied by different weights than the other

Encoder

ex:

- 1- query: the start (input)
- 2- key : keys are in the text you wrote
- 3- value : the result (output)



What is Self-attention?

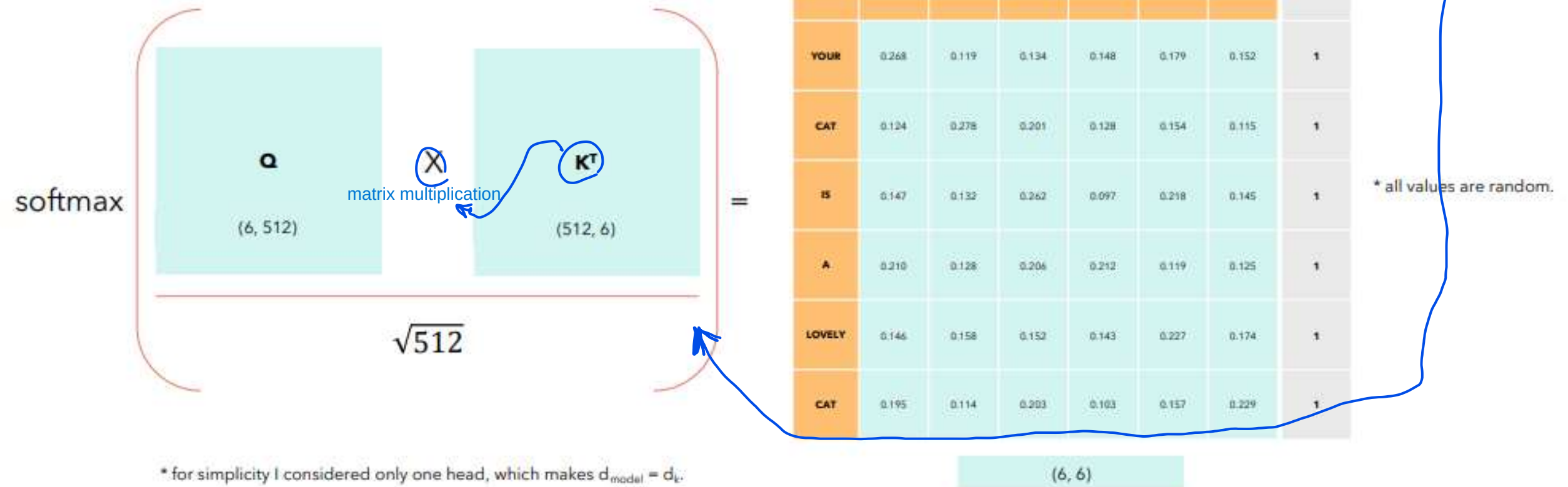
the relationships between the words

(Self-Attention allows the model to relate words to each other.)

In this simple case we consider the length of the sentence $\text{seq} = 6$ and length of the vector $\mathbf{d}_{\text{model}} = \mathbf{d}_k = 512$.

The matrices $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ are just the input sentence.

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \left(\text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V} \right)$$



How to Compute Self-attention?

6x6

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	-0.278	0.201	-0.128	0.154	-0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	-0.128	0.206	-0.212	0.119	-0.125
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174
CAT	0.195	-0.114	0.283	-0.103	0.157	-0.229

(6, 6)

X

V

(6, 512)

=

Attention

(6, 512)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Each row in this matrix captures not only the meaning (given by the embedding) or the position in the sentence (represented by the positional encodings) but also each word's interaction with other words.

Self-Attention properties

if i changed the position of the vector that wont effect the result
(because we already have position embedding)

- 1-• Self-Attention is permutation invariant
- 2-• Self-Attention requires no parameters. Up to now the interaction between words has been driven by their embedding and the positional encodings. This will change later
- 3-• We expect values along the diagonal to be the highest as correlation matrix
- 4-• If we don't want some positions to interact, we can always set their values to $-\infty$ before applying the softmax in this matrix and the model will not learn those interactions. We will use this in the decoder.

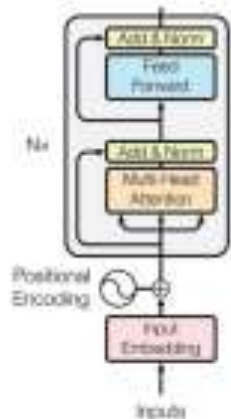
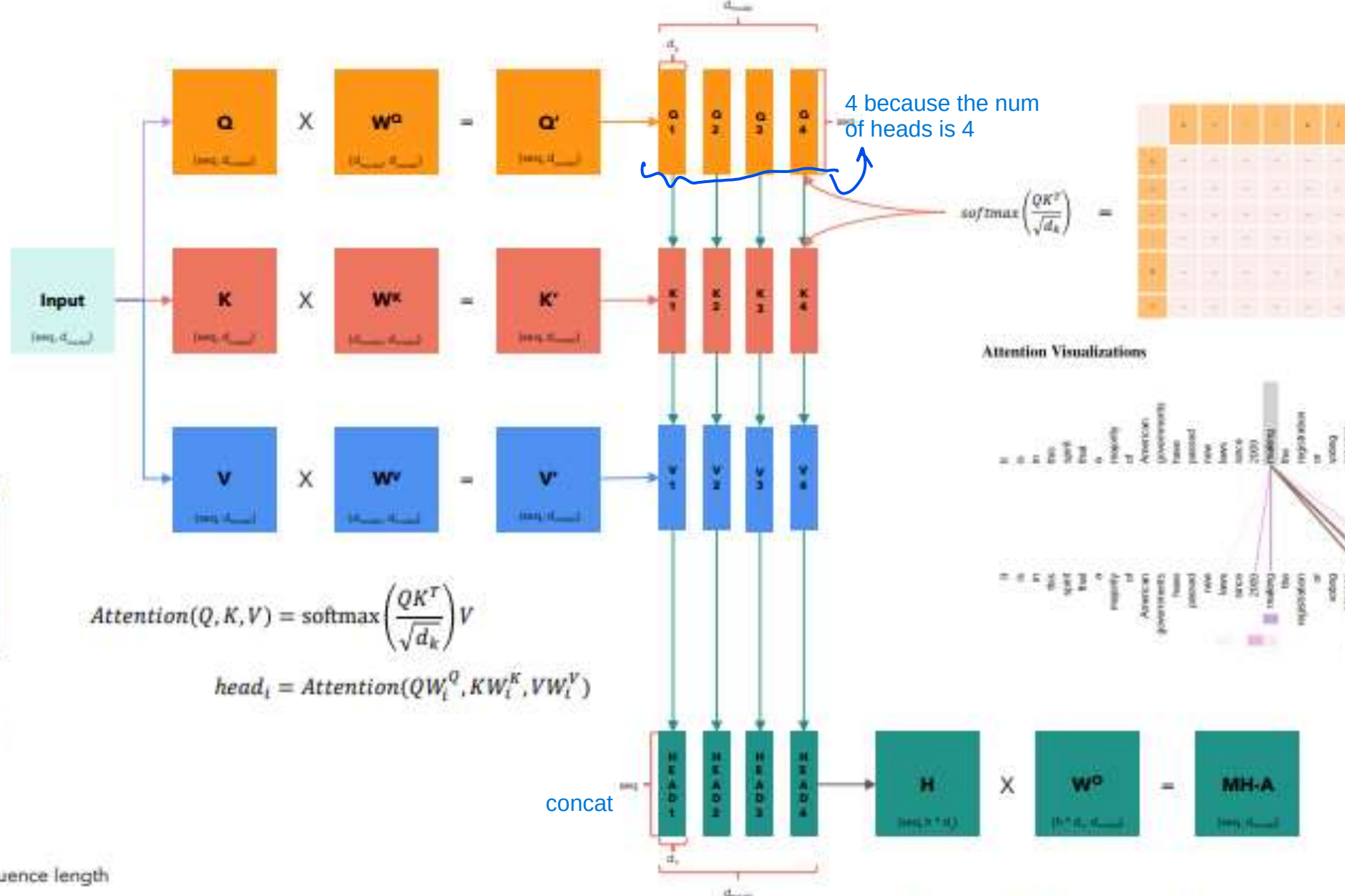
	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	0.278	0.201	0.128	0.154	0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	0.119	0.125
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174
CAT	0.195	0.114	0.203	0.103	0.157	0.229

Multi-head Attention

$$\text{Attention}(Q, K, V) = \left(\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \right) \text{ : head}$$

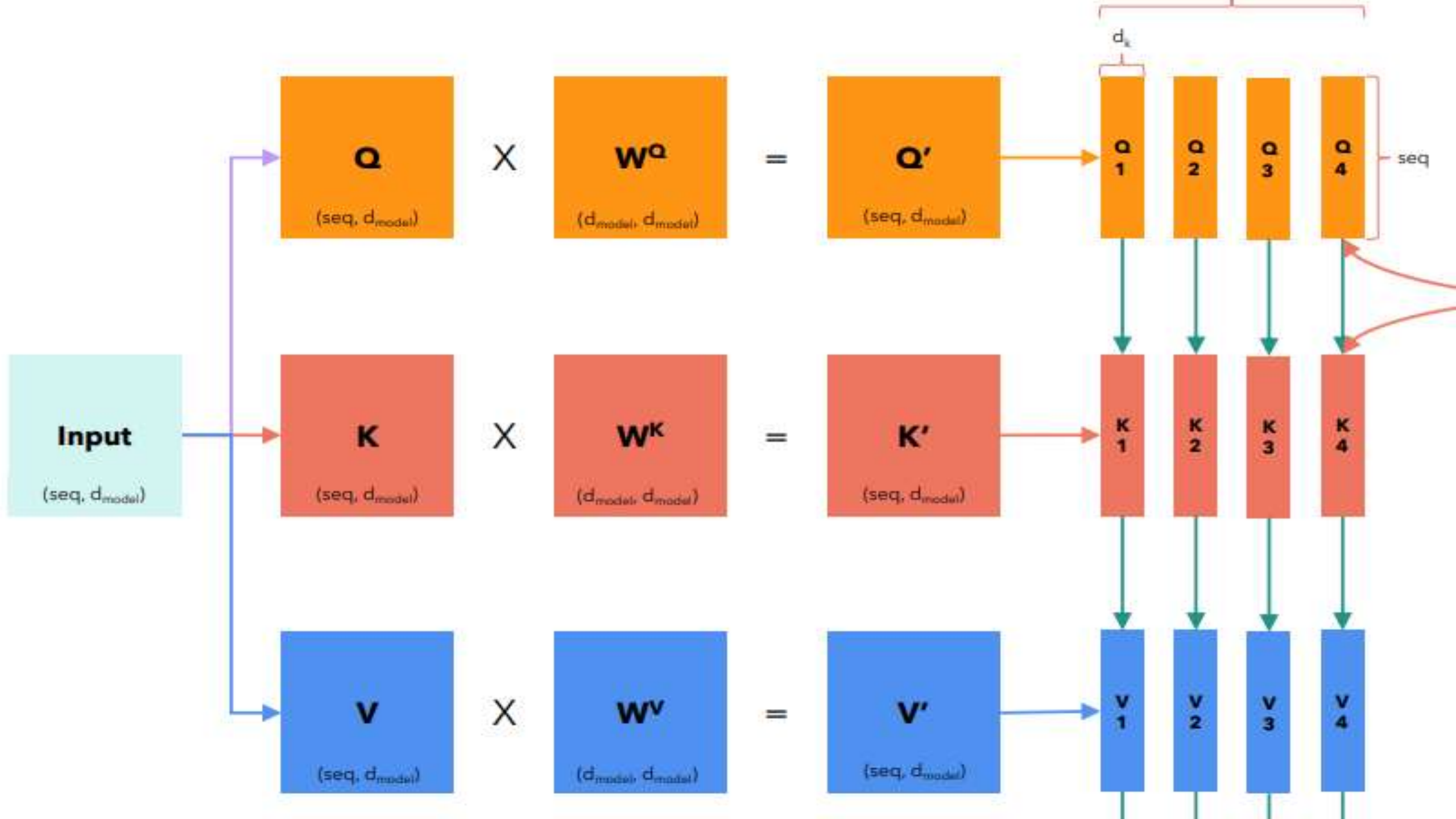
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1 \dots \text{head}_h) W^O$$

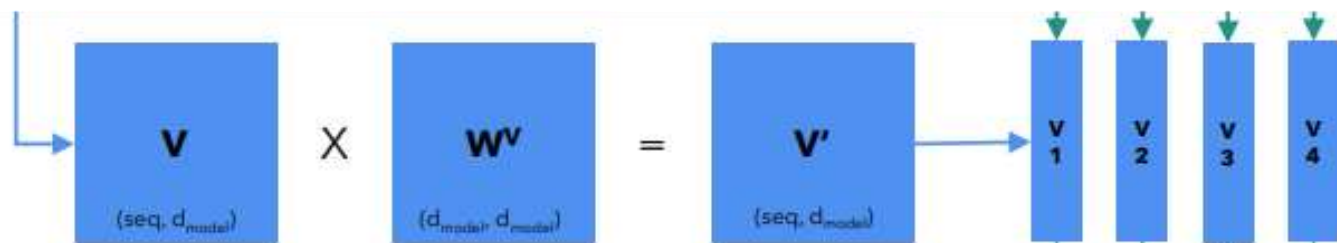
$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$



seq = sequence length
 d_{model} = size of the embedding vector
 h = number of heads
 $d_k = d_v$ = d_{model} / h

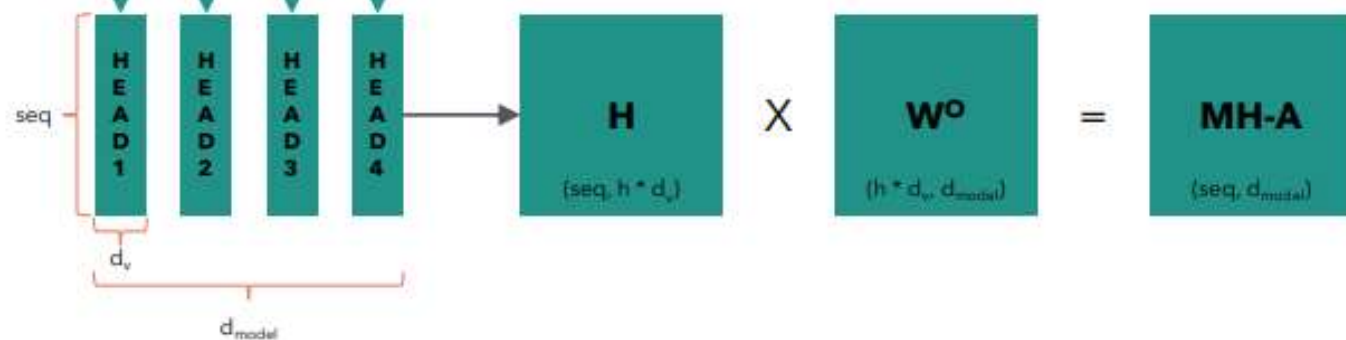
$$MultiHead(Q, K, V) = \text{Concat}(head_1 \dots head_h)W^O$$



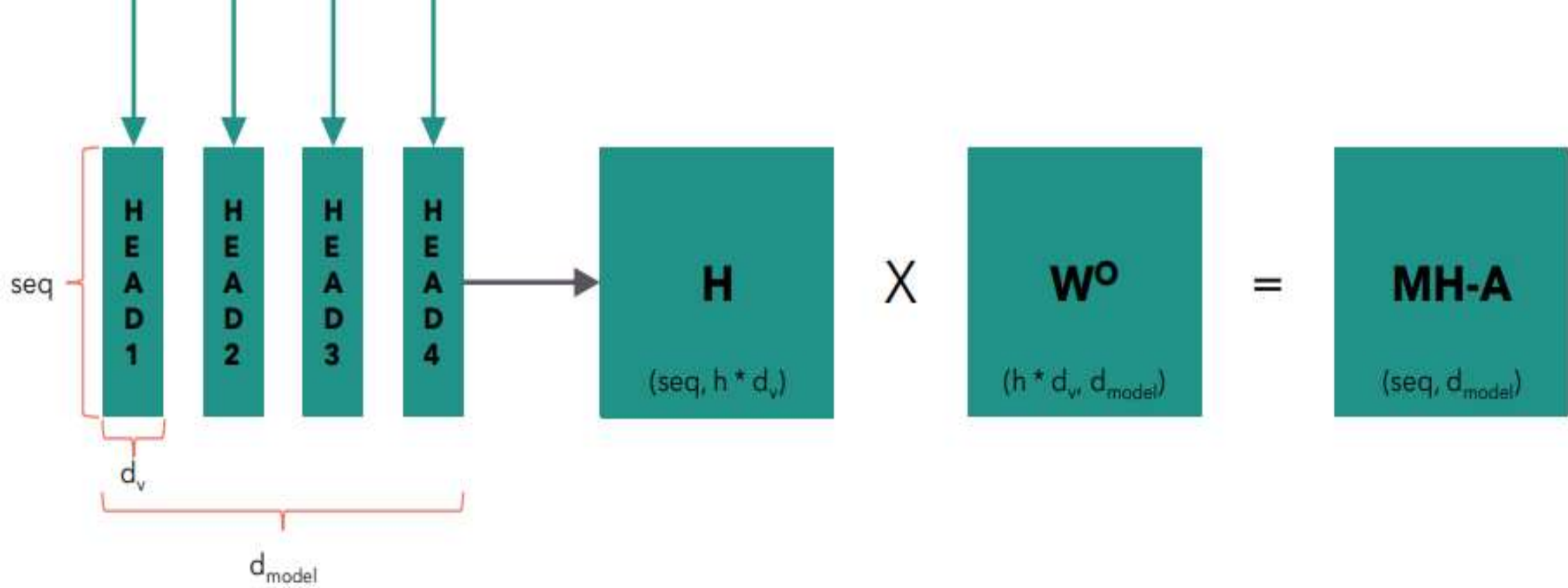


$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$



$$MultiHead(Q, K, V) = \text{Concat}(head_1 \dots head_h)W^o$$



$$MultiHead(Q, K, V) = Concat(head_1 \dots head_h)W^O$$

seq = sequence length
 d_{model} = size of the embedding vector
 h = number of heads
 $d_k = d_v$ = d_{model} / h

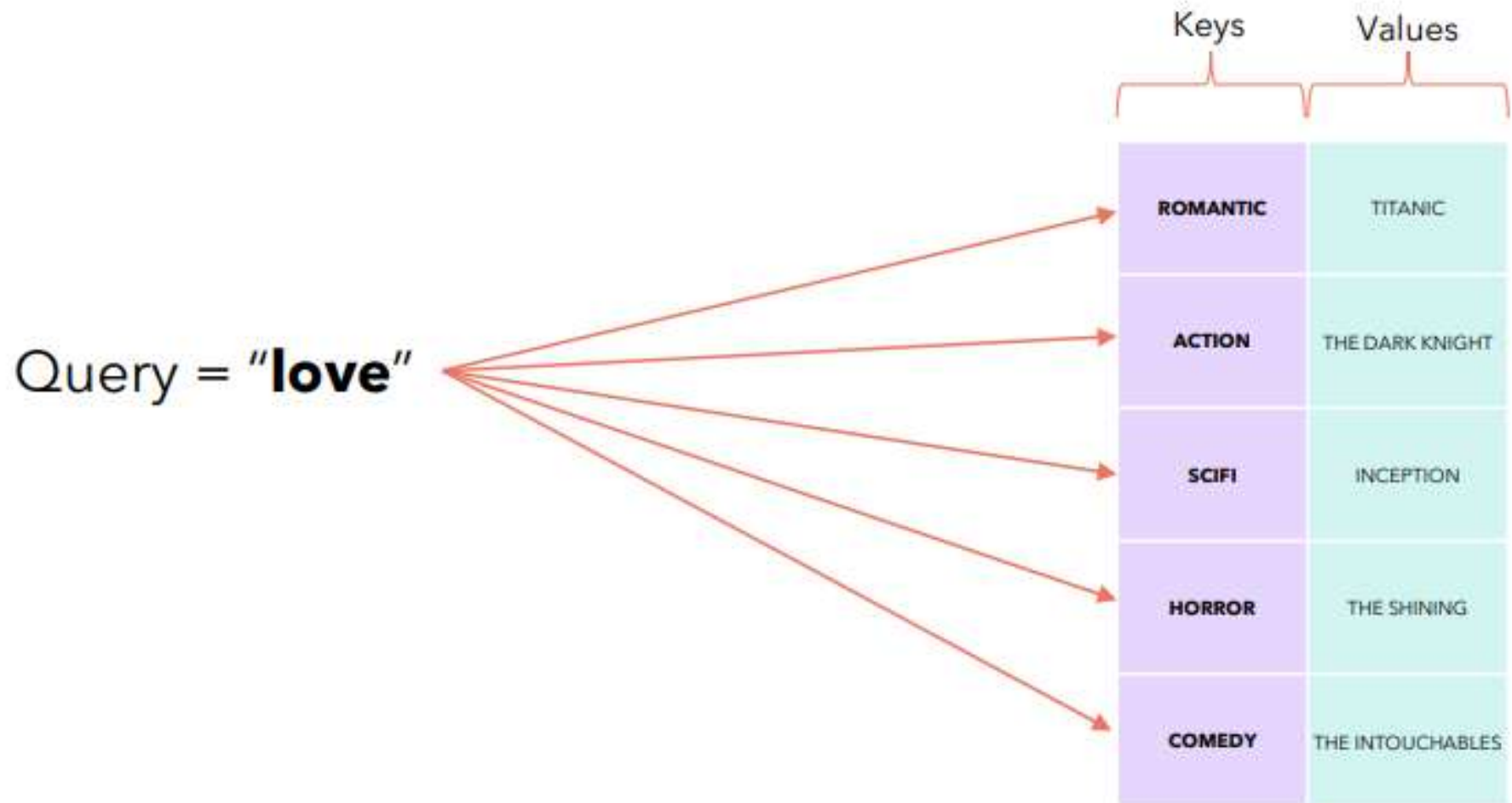
Why Multi-head Self-Attention Mechanism?

- 1- • **Capturing Different Types of Information:** Different attention heads can focus on different parts of the input. For example, (one head might focus on the subject of a sentence, while another might focus on the object.) By having multiple attention heads, the model can simultaneously attend to various parts of the input, capturing diverse types of information.
- 2- • **Enhanced Representation Learning:** By allowing the model to focus on different parts of the input simultaneously, multi-head attention helps in capturing richer representations. This capability enables the model to understand relationships between tokens at different levels of granularity, leading to better feature extraction and representation learning.
- 3- • **Reduction of Overfitting:** Introducing multiple attention heads can be seen as a form of ensemble learning within a single model. Each attention head can potentially learn different patterns or features from the data. This diversity can help in reducing overfitting, as the model is less likely to rely heavily on specific features or patterns.

Why Multi-head Self-Attention Mechanism?

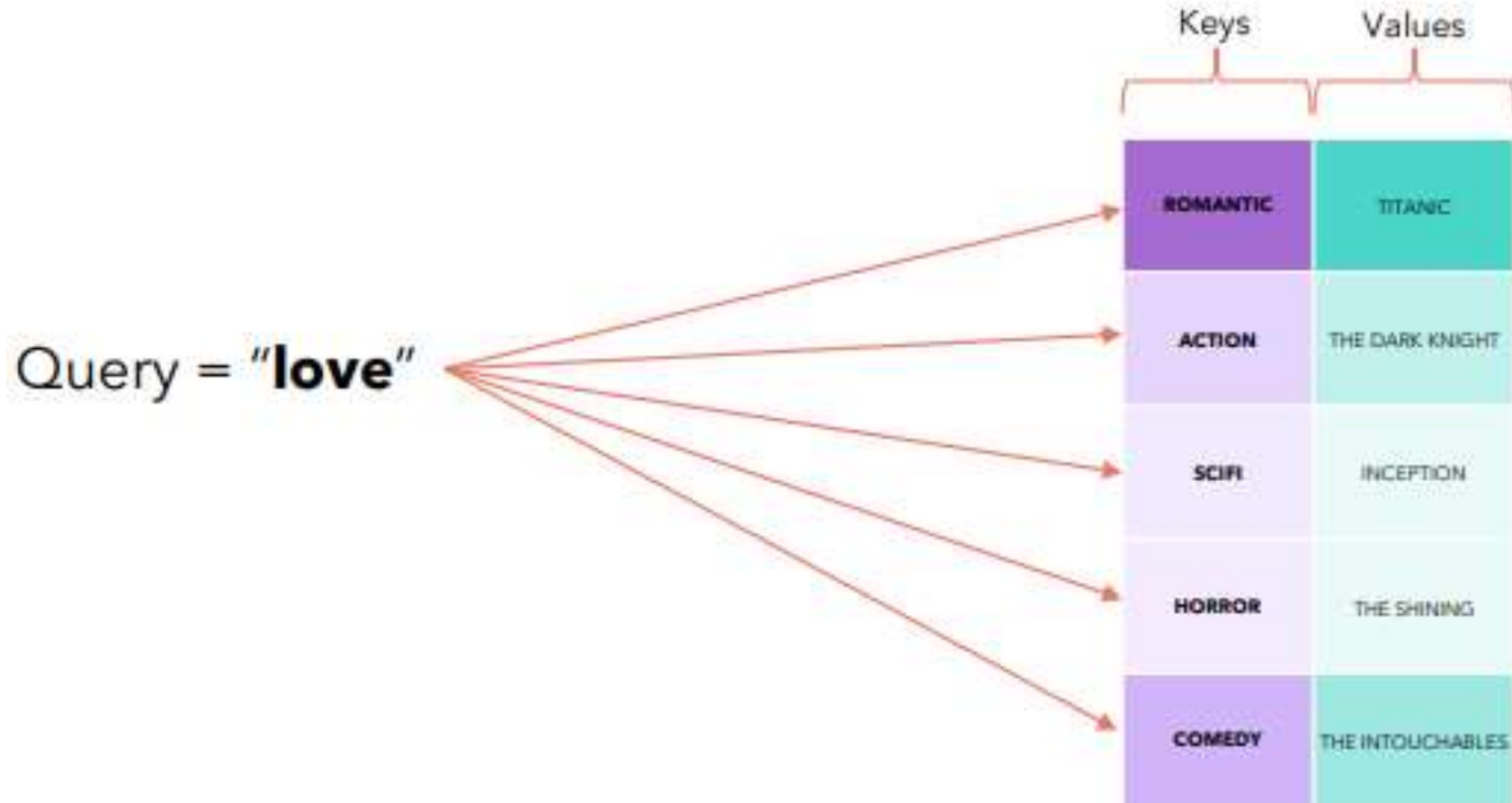
- 4- • **Parallelization:** While the concept of attention is inherently sequential, the computations within each attention head can be parallelized. This parallelization allows for efficient training and inference on hardware accelerators like GPUs, leading to faster training times.
- 5- • **Improved Generalization:** Multi-head attention enables the model to generalize better across different tasks and domains. By learning multiple sets of attention weights, the model can adapt its attention mechanism to the specific requirements of different tasks, leading to improved performance on a wide range of tasks without task-specific modifications.
- 6- • **Flexibility and Adaptability:** The ability to use multiple attention heads provides flexibility to the model, allowing it to adapt its attention mechanism based on the complexity and nature of the input data. This adaptability is particularly beneficial for handling long sequences or sequences with varying levels of complexity.

Why Query, Key and Value?



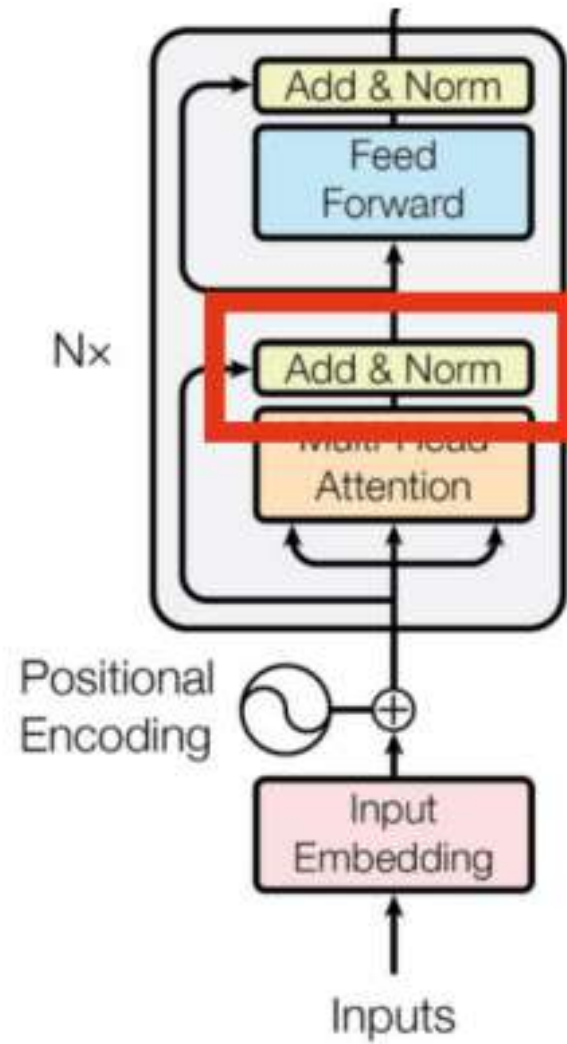
* this could be a Python dictionary or a database table.

Why Query, Key and Value?



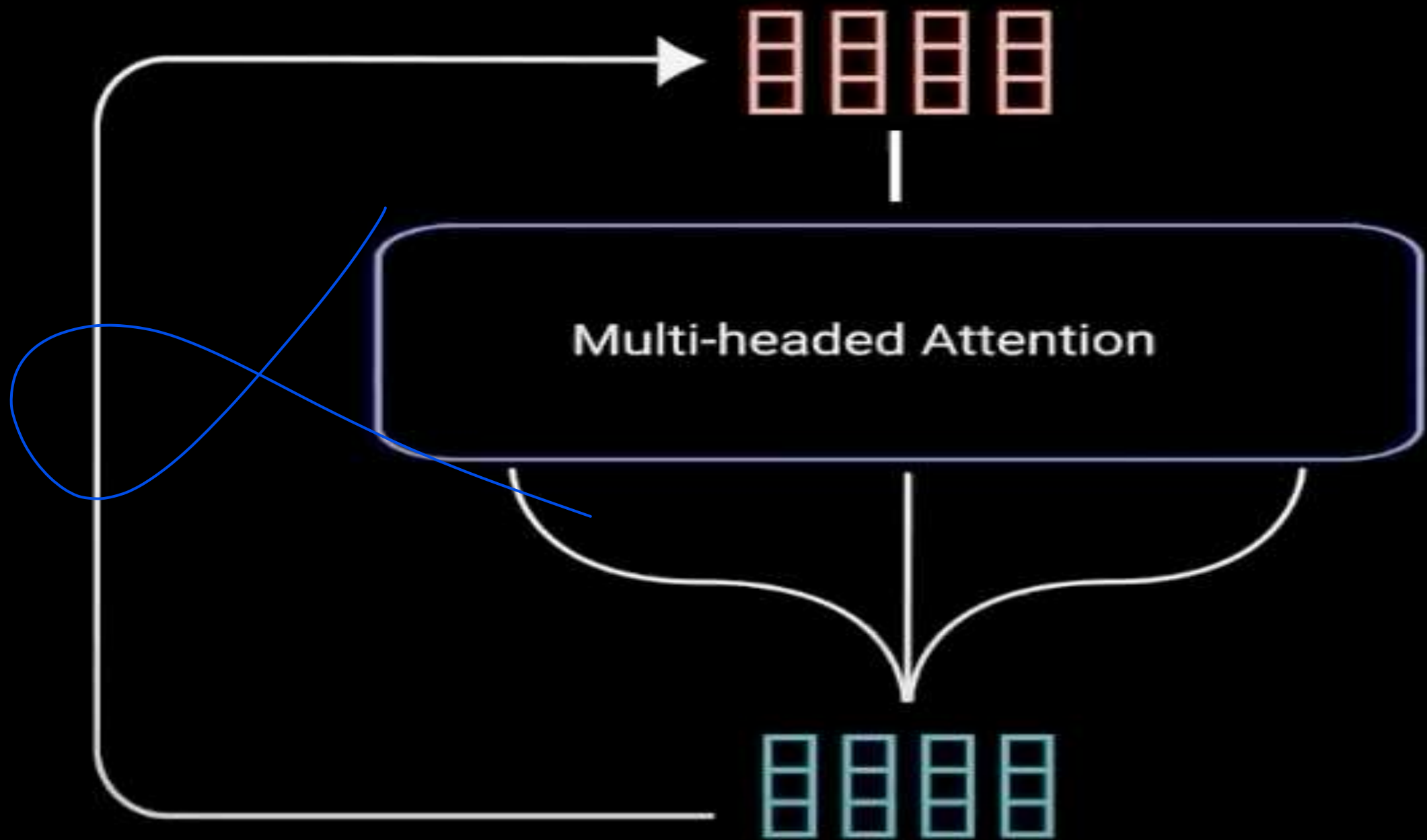
* this could be a Python dictionary or a database table.

Encoder



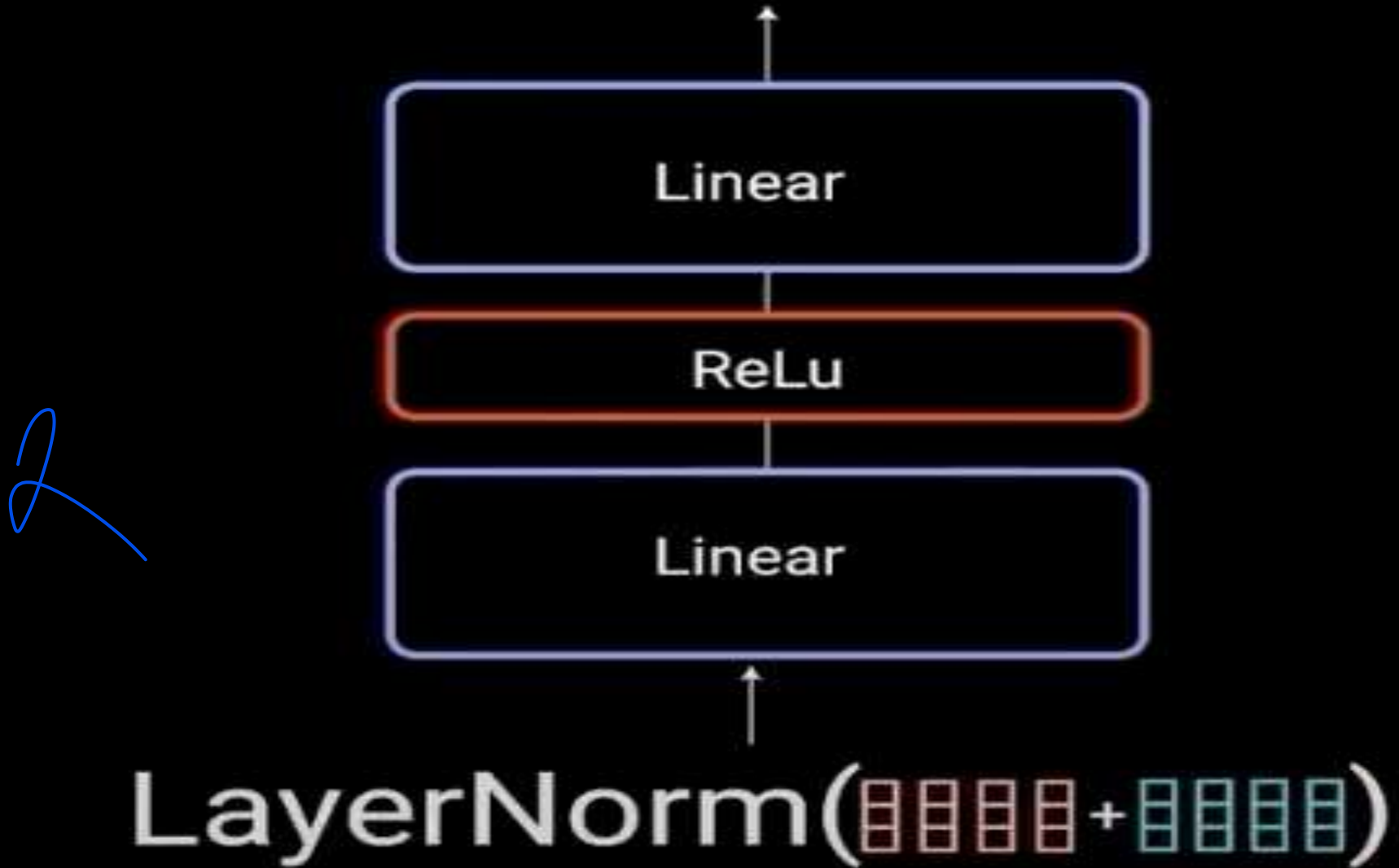
① The Residual Connections, Layer Normalization

- The multi-headed attention output vector is added to the original positional input embedding.
- This is called a residual connection.
- The output of the residual connection goes through a layer normalization.
- The residual connections help the network train, by allowing gradients to flow through the networks directly.
- The layer normalizations are used to stabilize the network which results in substantially reducing the training time necessary.



② Feed Forward Network feature extraction

- The normalized residual output gets projected through a pointwise feed-forward network for further processing.
- The pointwise feed-forward network is a couple of linear layers with a ReLU activation in between.
- The output of that is then again added to the input of the pointwise feed-forward network and further normalized.



Encoder Wrap-up

- All of these operations are to encode the input to a continuous representation with attention information.
- This will help the decoder focus on the appropriate words in the input during the decoding process.
- We can stack the encoder N times to further encode the information, where each layer has the opportunity to learn different attention representations therefore potentially boosting the predictive power of the transformer network.

compress the data

we can repeat the encoder and the layers more than one time

Decoder

4. Decoder layers

- 1-• The output of encoder is fed into the decoder layers next.
- 2-• The decoder has a similar sub-layer as the encoder. it has (two¹⁻ multi-headed attention layers) (a pointwise feed-forward layer) and (residual connections)³⁻ and (layer normalization after each sub-layer)⁴⁻.
- 3-• These sub-layers behave similarly to the layers in the encoder but each multi-headed attention layer has a different job.
- 4-• The decoder is capped off with a linear layer that acts as a classifier, and a softmax to get the word probabilities.

Decoder

- The decoder is autoregressive, it begins with a start token, and it takes in a list of previous outputs as inputs, as well as the encoder outputs that contain the attention information from the input.
- imp • The decoder stops decoding when it generates a token as an output.

Hi,



how



are



you?



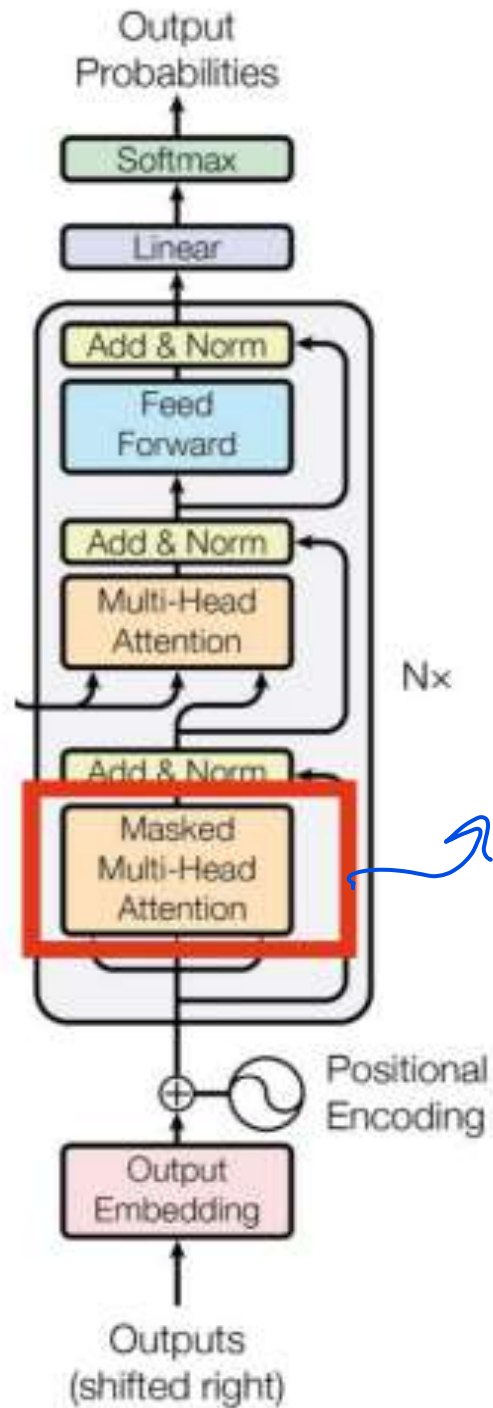
Transformers
Decoder



<start>

Decoder

transformers are supervised learning



Decoding Steps

- The beginning of the decoder is pretty much the same as the encoder.
- The input goes through an embedding layer and positional encoding layer to get positional embeddings.
- The positional embeddings get fed into the first multi-head attention layer which computes the attention scores for the decoder's input.

First Multi-Headed Attention

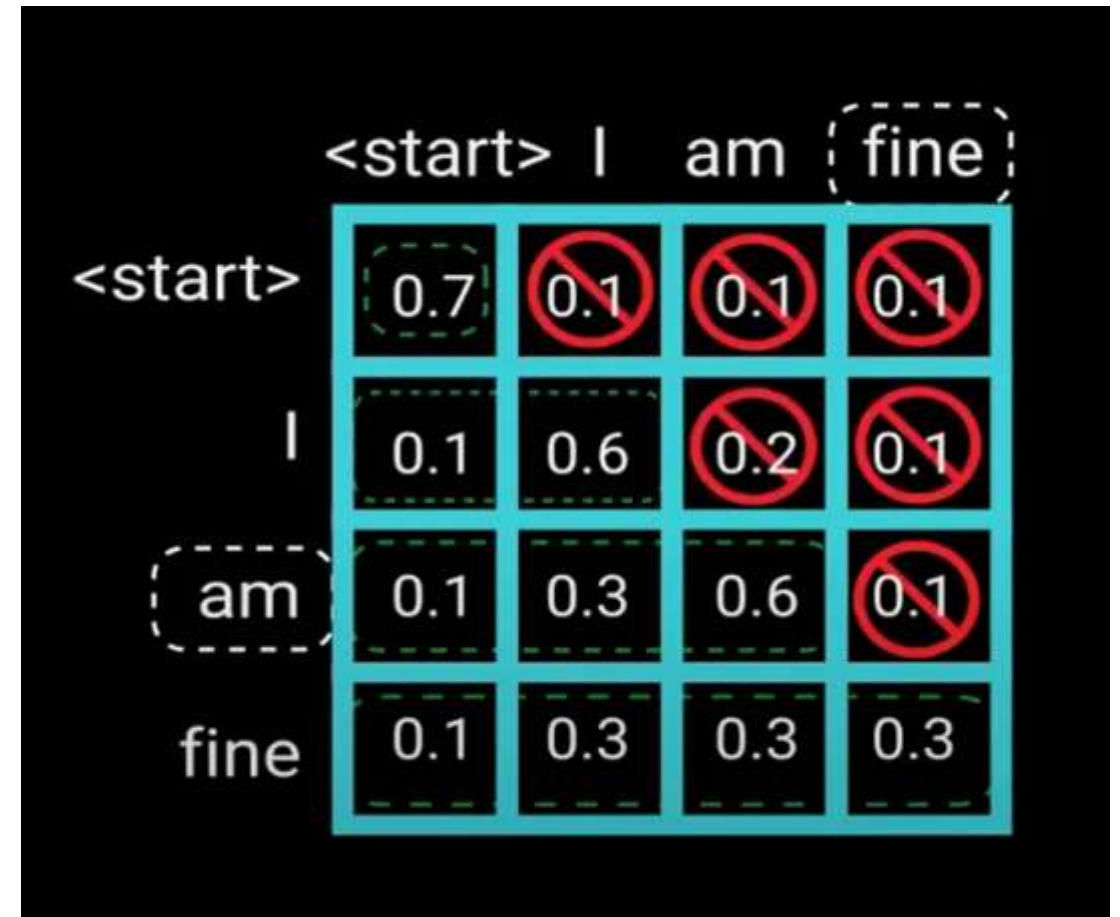
- This multi-headed attention layer operates slightly differently.
- Since the decoder is autoregressive and generates the sequence word by word, you need to prevent it from conditioning to future tokens.

why it should be masked :

a masked self-attention mechanism is used to ensure that each position in the output sequence attends only to positions before it in the sequence. This is necessary to prevent information leakage from future positions, which is crucial during the autoregressive generation of sequences.

Example

- For **example**, when computing attention scores on the word “am”, you should not have access to the word “fine”, because that word is a future word that was generated after.
- The word “am” should only have access to itself and the words before it. This is true for all other words, where they can only attend to previous words.



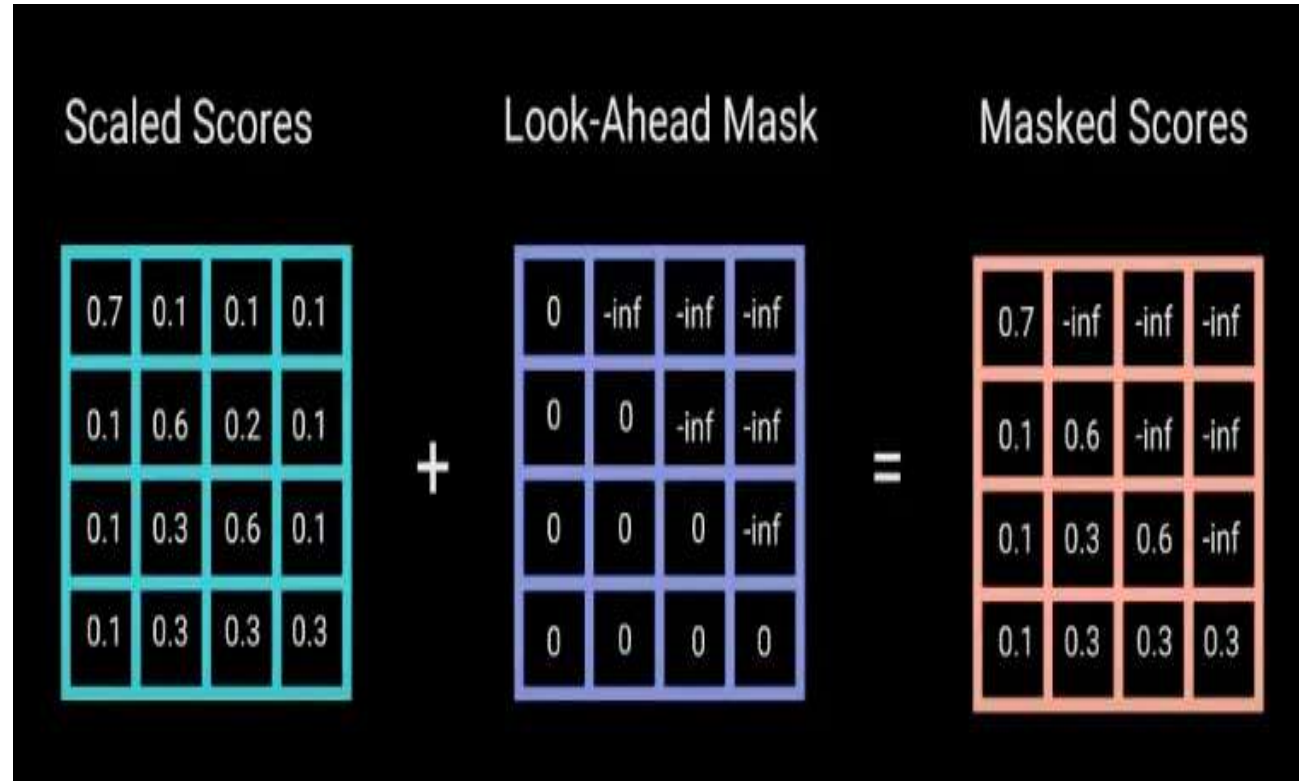
Masked Multi-Headed Attention

- We need a method to prevent computing attention scores for future words. This method is called masking.
- To prevent the decoder from looking at future tokens, you apply a look ahead mask.
- The mask is added before calculating the softmax, and after scaling the scores.



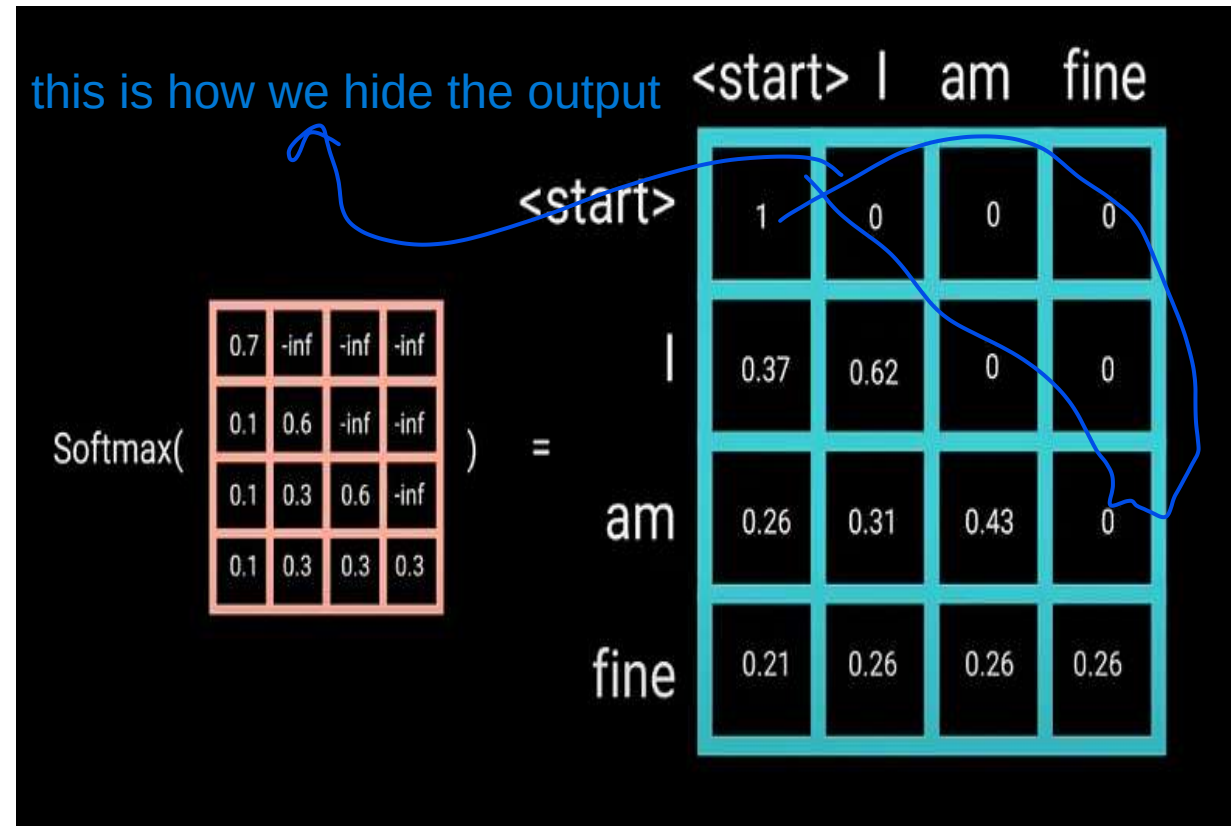
Look-Ahead Mask technique

- The mask is a matrix that's the same size as the attention scores filled with values of 0's and negative infinities.
- When you add the mask to the scaled attention scores, you get a matrix of the scores, with the top right triangle filled with negativity infinities.



Look-Ahead Mask

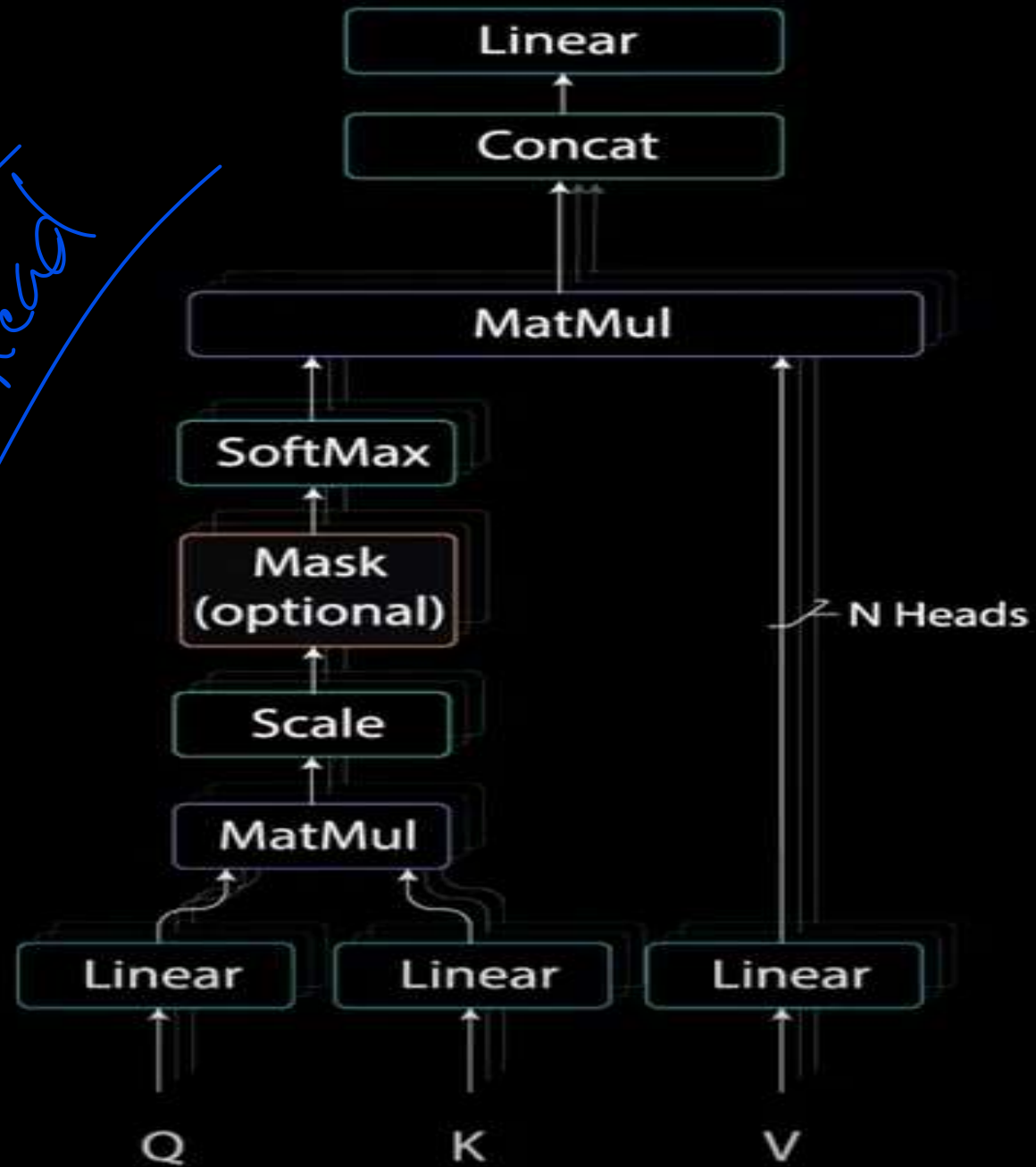
- The reason for the mask is because once you take the softmax of the masked scores, the negative infinities get zeroed out, leaving zero attention scores for future tokens.
- As you can see in the figure below, the attention scores for “am”, has values for itself and all words before it but is zero for the word “fine”. This essentially tells the model to put no focus on those words.



First Multi-Headed Attention

- This masking is the only difference in how the attention scores are calculated in the first multi-headed attention layer.
- This layer still has multiple heads, that the mask is being applied to, before getting concatenated and fed through a linear layer for further processing.
- The output of the first multi-headed attention is a masked output vector with information on how the model should attend on the decoder's input.

Masked Multi-head

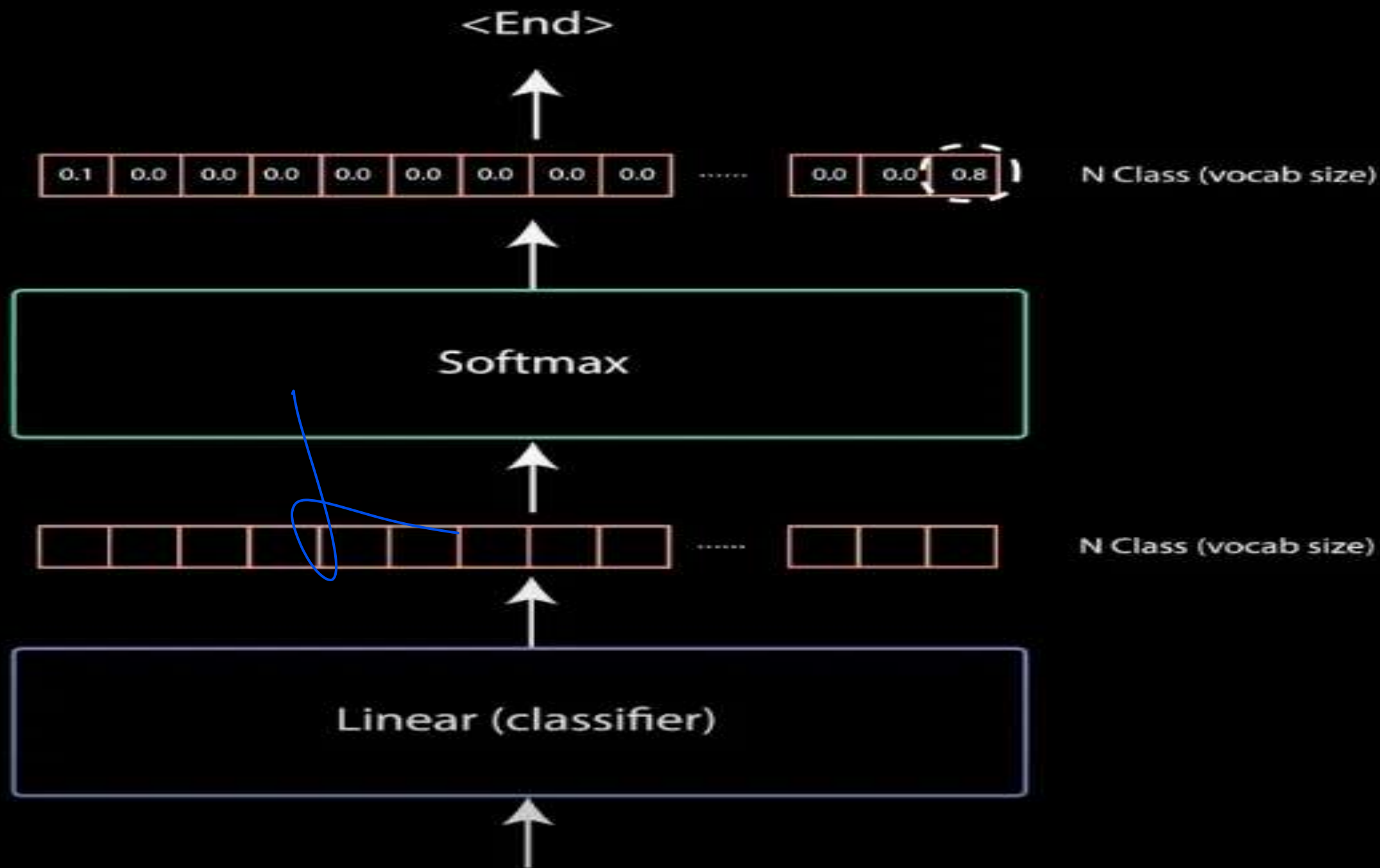


Second Multi-Headed Attention

- For this layer, the encoder's outputs are the queries and the keys, and the first multi-headed attention layer outputs are the values.
- This process matches the encoder's input to the decoder's input, allowing the decoder to decide which encoder input is relevant to put a focus on.
- The output of the second multi-headed attention goes through a pointwise feedforward layer for further processing.

The Output

- The output of the final pointwise feedforward layer goes through a final linear layer, that acts as a classifier.
- The classifier is as big as the number of classes you have.
- For example, if you have 10,000 classes for 10,000 words, the output of that classifier will be of size 10,000.
- The output of the classifier then gets fed into a softmax layer, which will produce probability scores between 0 and 1.
- We take the index of the highest probability score, and that equals our predicted word.



Decoder Outputs

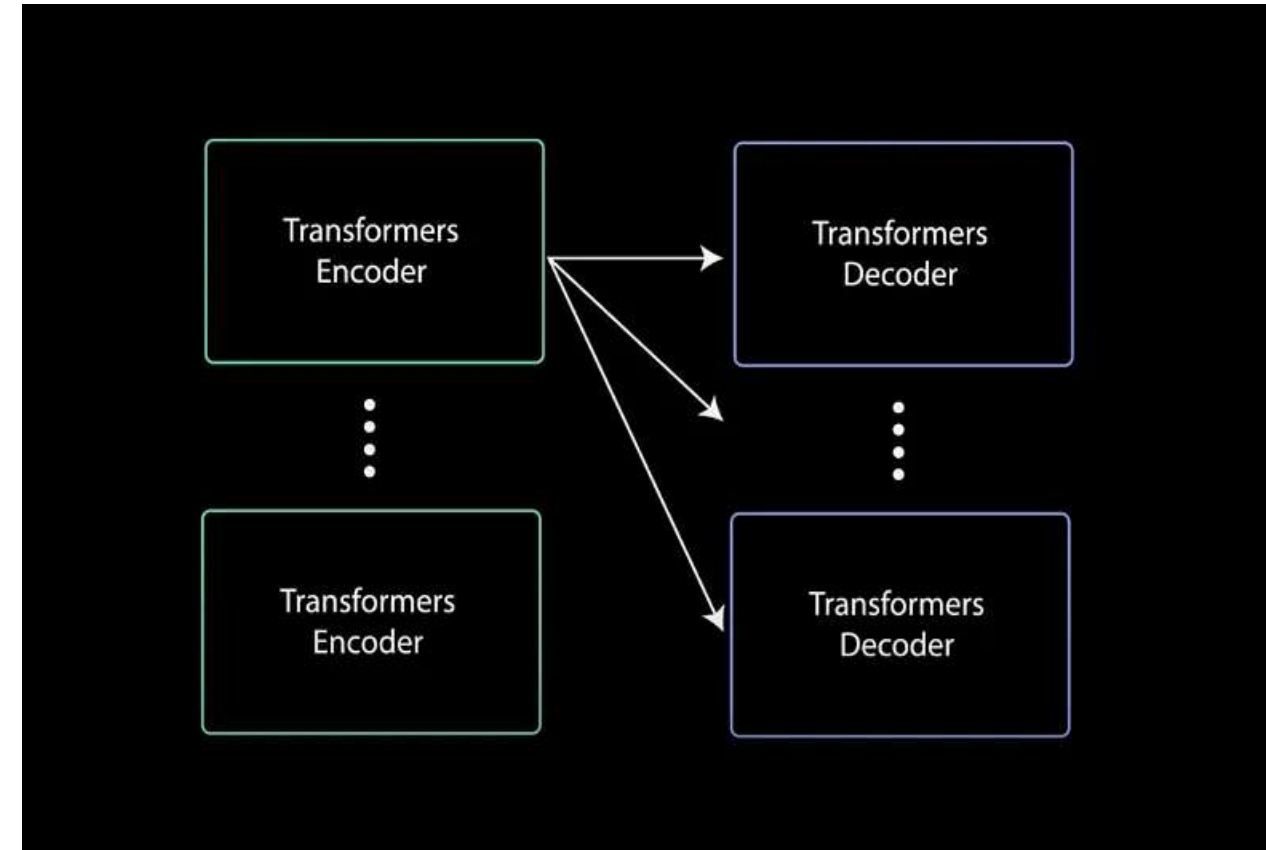
- We take the index of the highest probability score, and that equals our predicted word.
- For our case, the highest probability prediction is the final class which is assigned to the end token.

Decoder

we can use more than one

its not mirroring like the autoencoders

- The decoder can also be stacked N layers high, each layer taking in inputs from the encoder and the layers before it.
- By stacking the layers, the model can learn to extract and focus on different combinations of attention from its attention heads, potentially boosting its predictive power.



Training

- Transformers are trained using supervised learning.
- To train a model for translation tasks we need two sentences in different languages that are translations of each other.
- Once we have a lot of sentence pairs, we can start training our model.
- Let's say we want to translate English to Italian. Our encoded input will be an English sentence and the input for the decoder will be an Italian sentence.
- The model's predictions are compared with the correct target sequence, and optimization algorithms adjust the model's parameters to minimize the difference between predicted and correct outputs.
- This is done by going through the training data in batches and improving the model's performance.

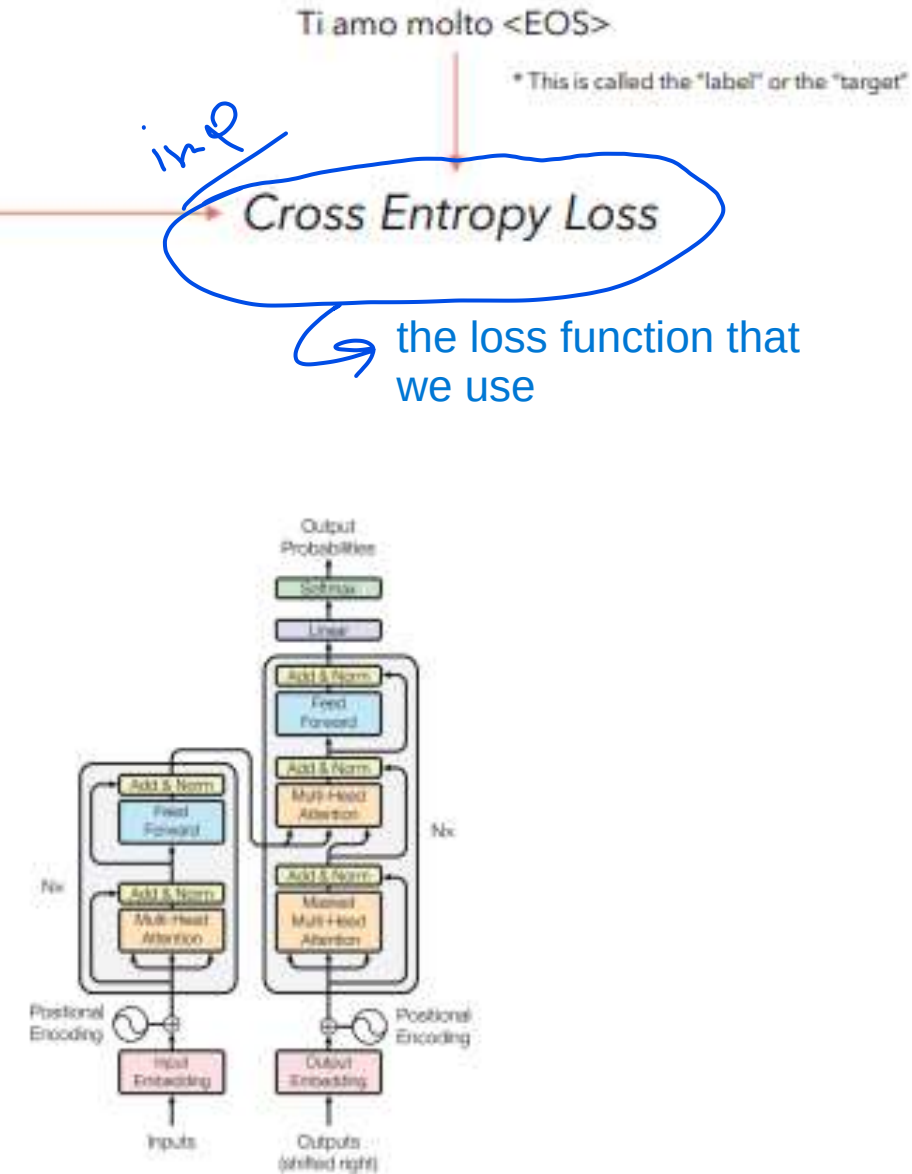
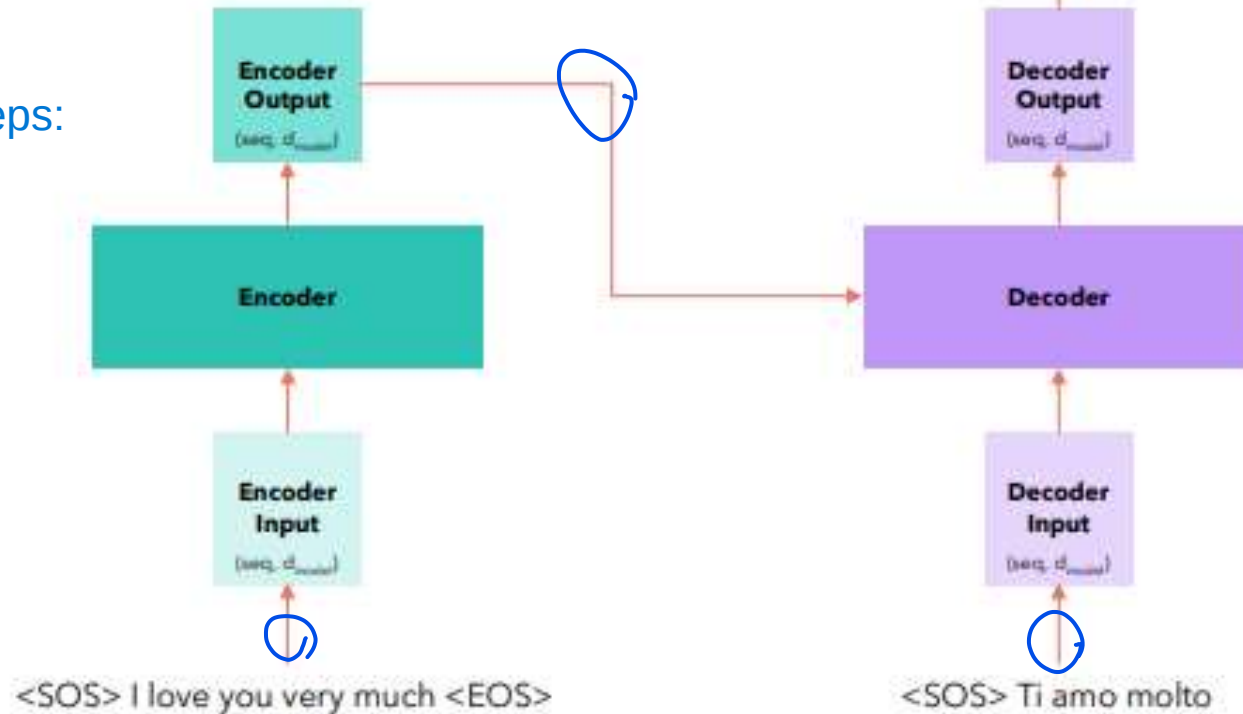
Training

Time Step = 1

It all happens in one time step!

The encoder outputs, for each word a vector that not only captures its meaning (the embedding) or the position, but also its interaction with other words by means of the multi-head attention.

steps:



We prepend the $\langle \text{SOS} \rangle$ token at the beginning. That's why the paper says that the decoder input is shifted right.

⌘ Inference

- Inferring with those models is different from the training, which makes sense because in the end we want to translate an English sentence without having the Italian sentence.
- The trick here is to re-feed our model for each position of the output sequence until we come across an end-of-sentence token.

Step by Step:

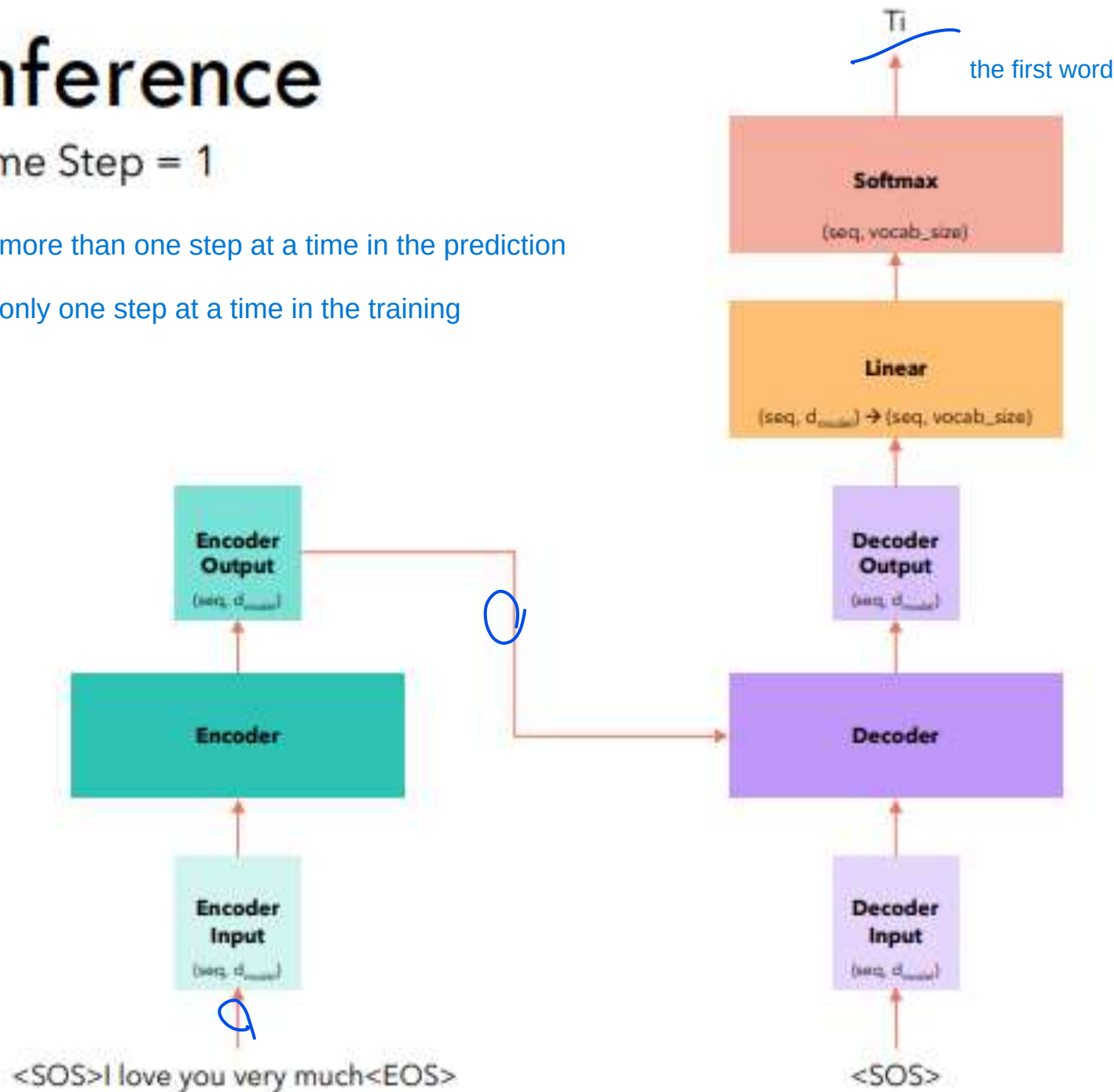
- 1-• Input the full encoder sequence (English sentence) and as decoder input, we take an (empty sequence) with only a start-of-sentence token on the first position. This will output a sequence where we will only take the first element.
- 2-• That element will be filled into second position of our decoder input sequence, which now has a start-of-sentence token and a first word/character in it.
- 3-• Input both the encoder sequence and the new decoder sequence into the model. Take the second element of the output and put it into the decoder input sequence.
- 4-• Repeat this until you predict an end-of-sentence token, which marks the end of the translation.

Inference

Time Step = 1

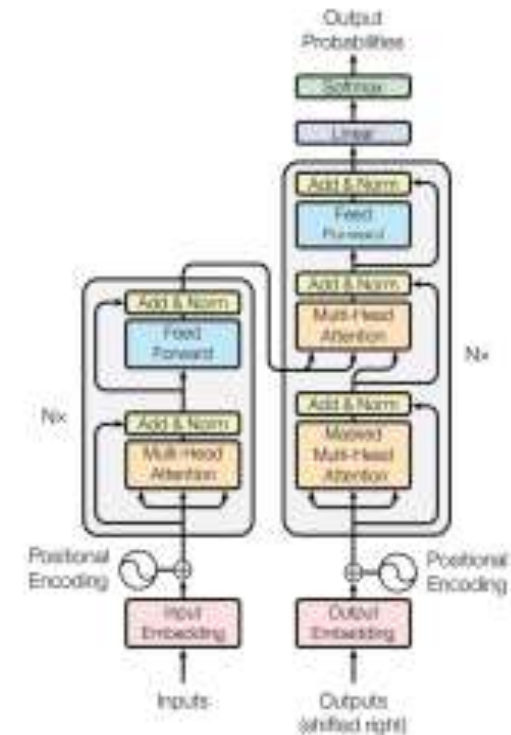
more than one step at a time in the prediction

only one step at a time in the training



We select a token from the vocabulary corresponding to the position of the token with the maximum value.

The output of the last layer is commonly known as **logits**



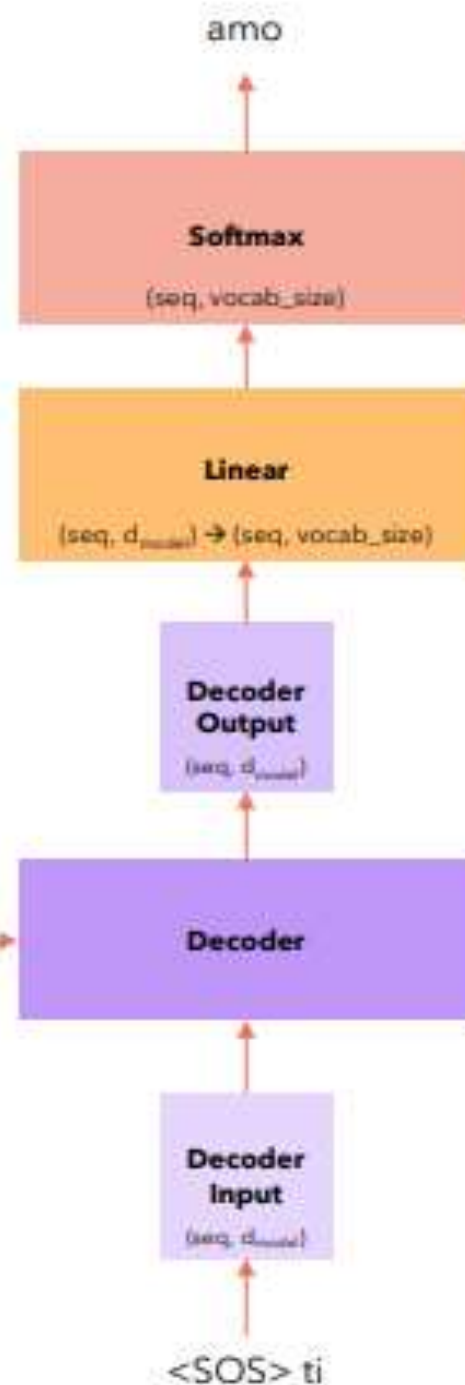
* Both sequences will have same length thanks to padding

Inference

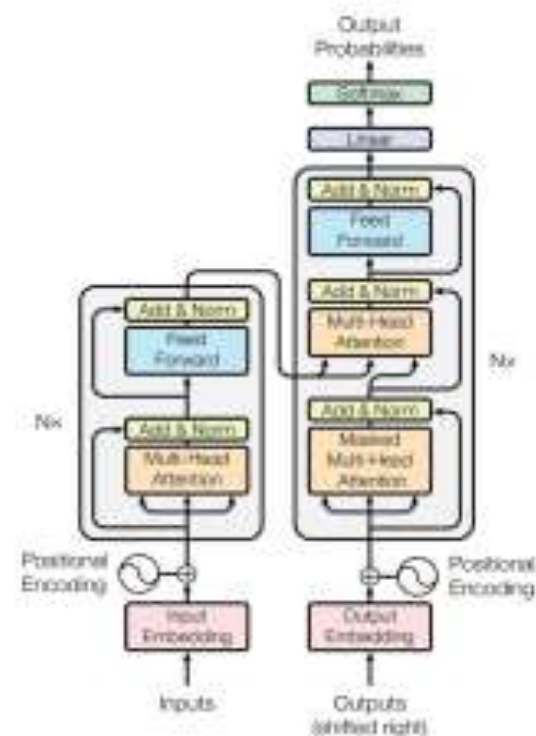
Time Step = 2

Use the encoder output from the first time step

<SOS>I love you very much<EOS>



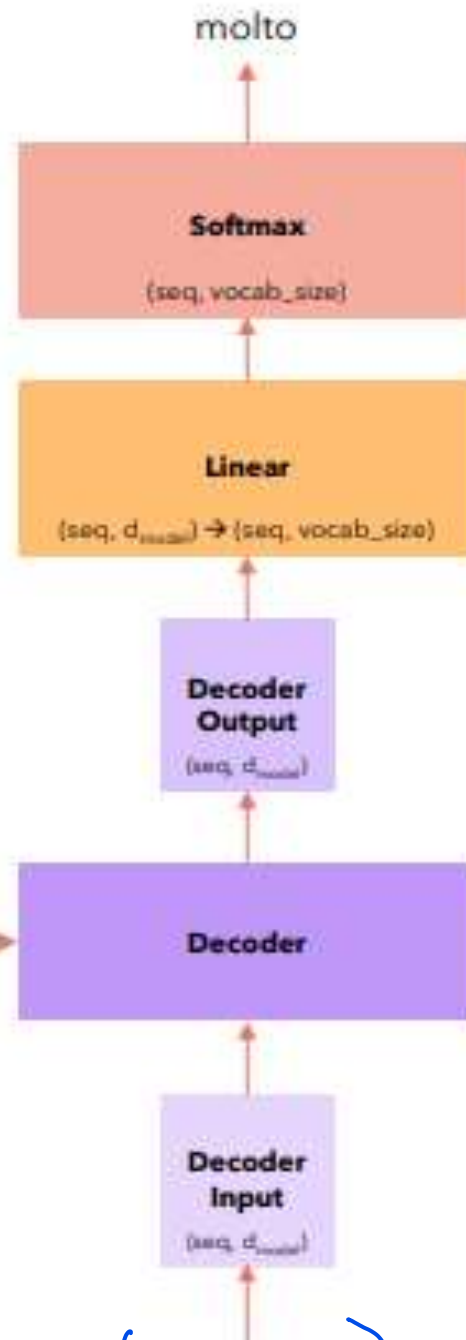
Since decoder input now contains **two** tokens, we select the softmax corresponding to the second token.



Append the previously output word to the decoder input

Inference

Time Step = 3

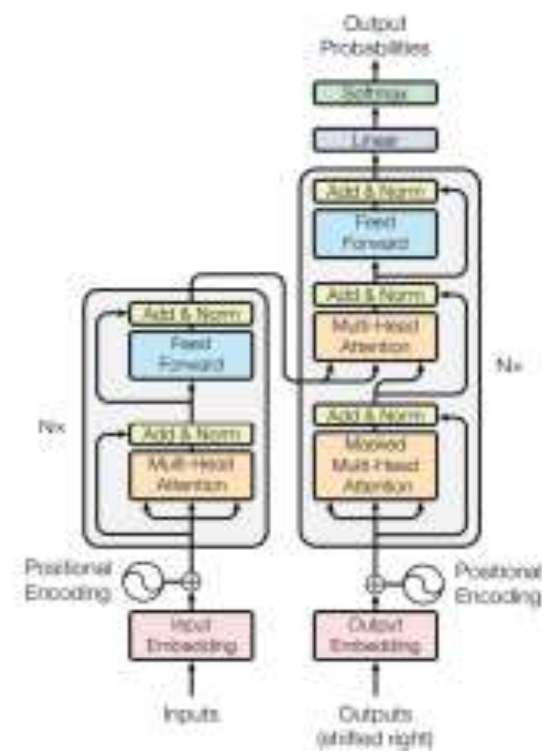


Since decoder input now contains **three** tokens, we select the softmax corresponding to the third token.

Use the encoder output from the first time step

<SOS>I love you very much<EOS>

<SOS> ti amo



Append the previously output word to the decoder input

Inference

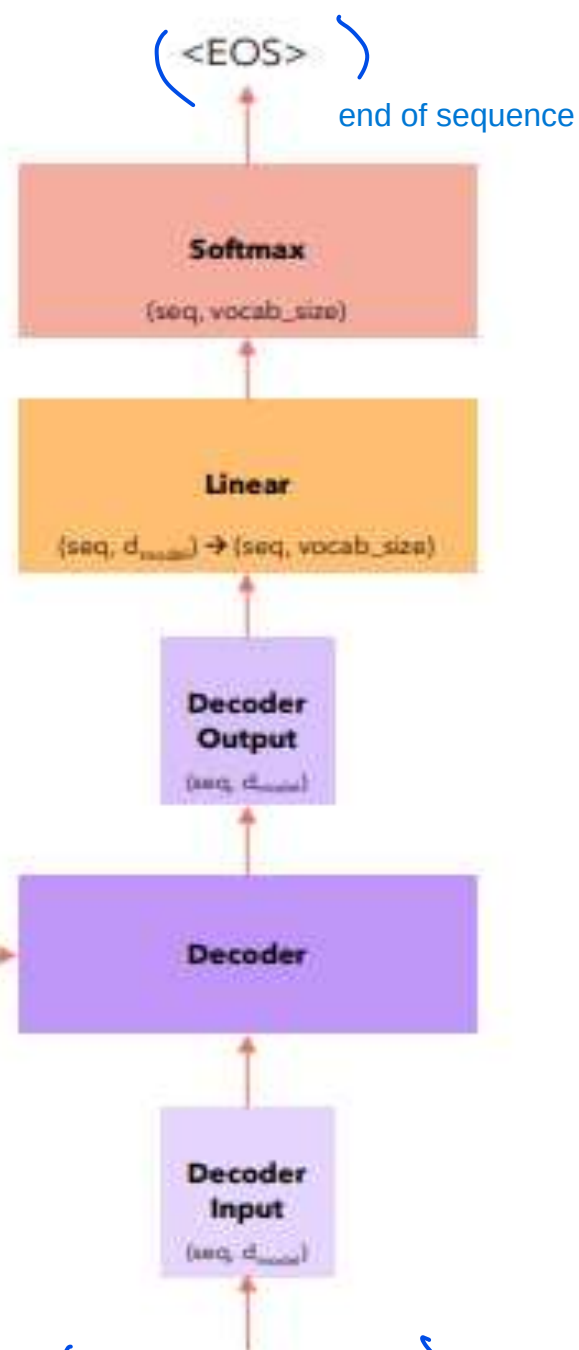
Time Step = 4

the prediction word by word

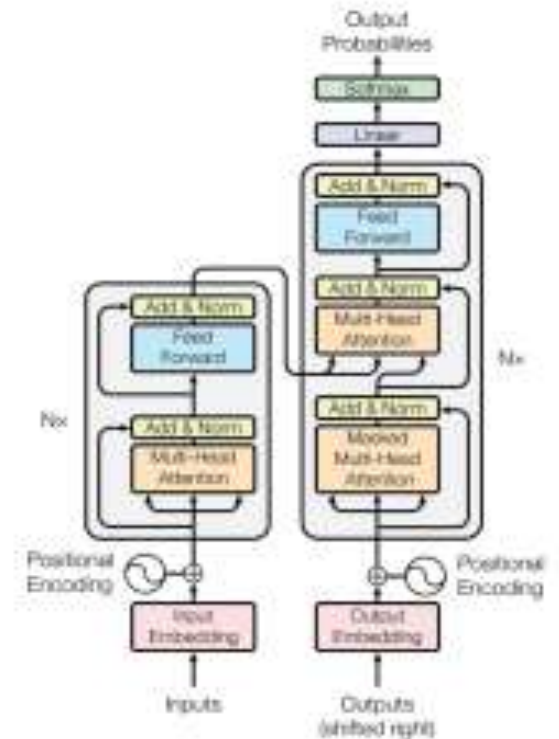
Use the encoder output from the first time step

<SOS>I love you very much<EOS>

in Put (<SOS> ti amo molto)



Since decoder input now contains **four** tokens, we select the softmax corresponding to the fourth token.



Append the previously output word to the decoder input

Types of transformer models

1-• Bidirectional transformers:

- Bidirectional encoder representations from transformers (BERT) models modify the base architecture to process words in relation to all the other words in a sentence rather than in isolation.
- Technically, it employs a mechanism called the bidirectional masked language model (MLM).
- During pretraining, BERT randomly masks some percentage of the input tokens and predicts these masked tokens based on their context.
- The bidirectional aspect comes from the fact that BERT takes into account both the left-to-right and right-to-left token sequences in both layers for greater comprehension.

Types of transformer models

2-• Generative pretrained transformers:

- GPT models use stacked transformer decoders that are pretrained on a large corpus of text by using language modeling objectives.
- They are autoregressive, (which means that they regress or predict the next value in a sequence based on all preceding values.)
- By using more than 175 billion parameters, GPT models can generate text sequences that are adjusted for style and tone.
- GPT models have sparked the research in AI toward achieving artificial general intelligence. This means that organizations can reach new levels of productivity while reinventing their applications and customer experiences.

Types of transformer models

3-• Bidirectional and autoregressive transformers:

- A bidirectional and auto-regressive transformer (BART) is a type of transformer model that combines bidirectional and autoregressive properties.
- It's like a blend of BERT's bidirectional encoder and GPT's autoregressive decoder. (BERT + GPT)
- It reads the entire input sequence at once and is bidirectional like BERT.
- However, it generates the output sequence one token at a time, conditioned on the previously generated tokens and the input provided by the encoder.

Types of transformer models

GPT4 ---> text to image

4-• Transformers for multimodal tasks:

- Multimodal transformer models such as ViLBERT and VisualBERT are designed to handle multiple types of input data, typically text and images.
- They extend the transformer architecture by using dual-stream networks that process visual and textual inputs separately before fusing the information.

5-• Vision transformers:

- Vision transformers (ViT) repurpose the transformer architecture for image classification tasks.
- Instead of processing an image as a grid of pixels, they view image data as a sequence of fixed-size patches, similar to how words are treated in a sentence.
- Each patch is flattened, linearly embedded, and then processed sequentially by the standard transformer encoder. Positional embeddings are added to maintain spatial information.

Main References

- <https://www.linkedin.com/pulse/introduction-transformer-models-neural-networks-machine-learning/>
- <https://www.youtube.com/watch?v=4Bdc55j80l8>
- <https://www.youtube.com/watch?v=bCz4OMemCcA&t=946s>

